



Redes de Computadoras I

Capítulo 3 – la capa de enlace de datos

Prof. Dr. Enrique Dávalos

Agenda

1.- Cuestiones de diseño de la capa de enlace de datos. Servicios proporcionados. Entramado

2.- Control de errores. Fundamento teórico

2.1 Detección de errores

a) Bits de paridad

b) Checksums

c) CRC

2.2 Corrección de errores

a) Código de Hamming

b) Códigos convolucionales



La capa de enlace de datos

Responsable por el envío de **TRAMAS** de información en un **enlace simple (siguiente salto)**

- Maneja errores de transmisión y regula el flujo de datos

Aplicación
Transporte
Red
Enlace
Física

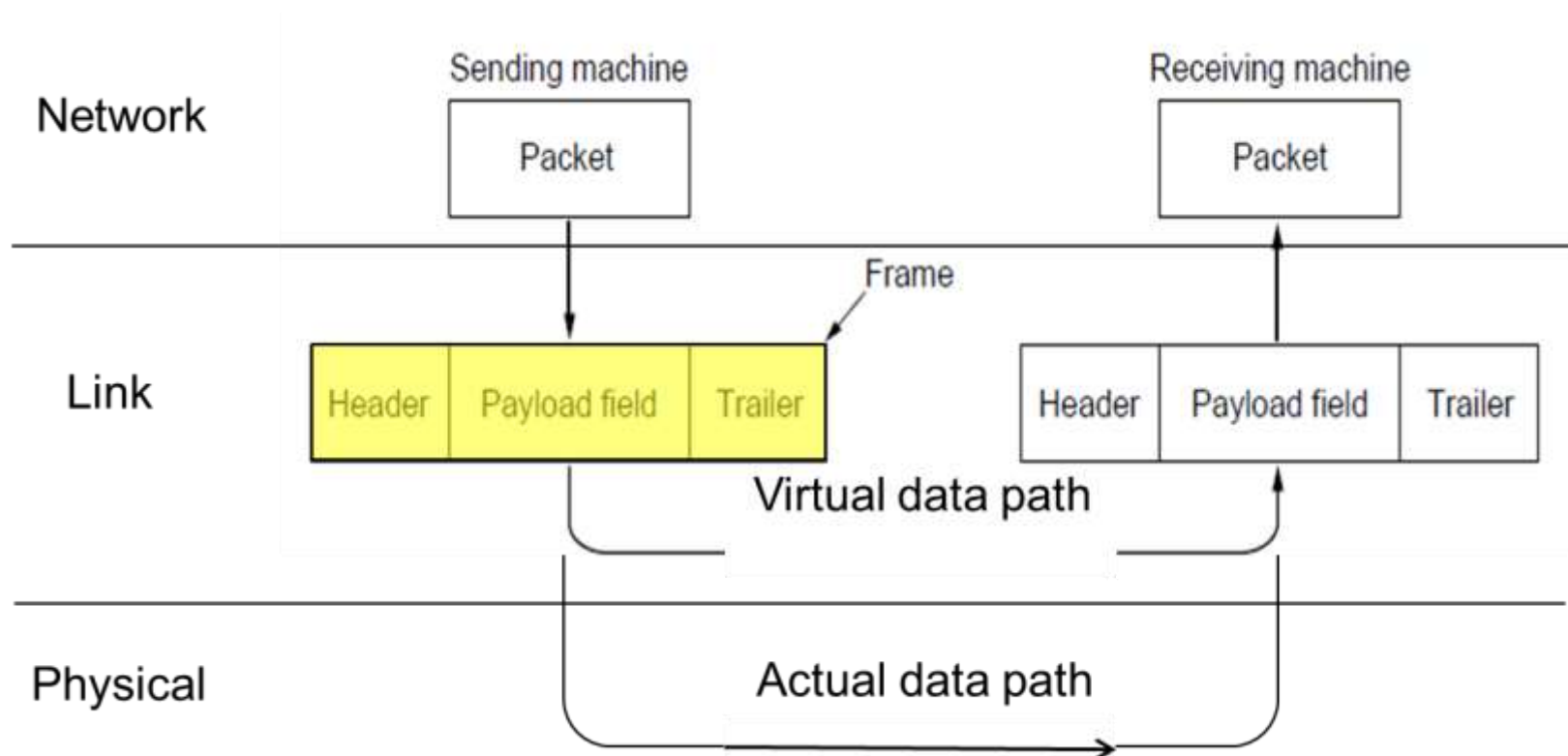


3.1 Cuestiones de diseño

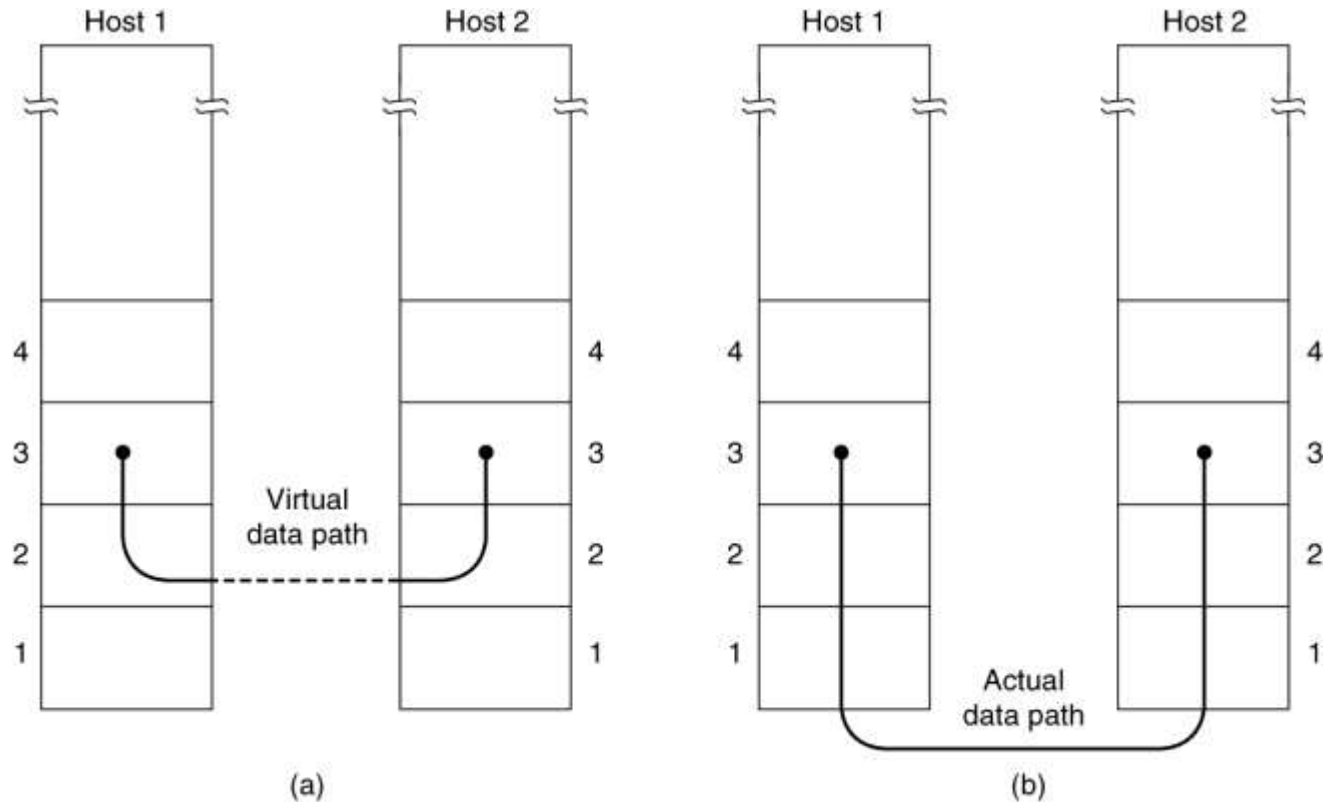
- Proporcionar una **interfaz de servicio** bien definida con la capa de red (3.1.1)
- **Identificación de las tramas** en la secuencia de bytes como segmentos autocontenidos (3.1.2)
- **Detección y Corrección de errores** de transmission (3.1.3)
- **Control de flujo:** No permitir que receptores más lentos sean sobrepasados por transmisores más rápidos (3.1.4)
- **Direccionamiento** a nivel del enlace de datos (*direccionamiento físico*) – inicio y fin del ENLACE

Tramas

La capa de enlace acepta PAQUETES de la capa de red, los encapsula en TRAMAS y los envía utilizando la capa física. La recepción es el proceso opuesto.



Servicios a la capa de red



El principal servicio es la transferencia de datos de la capa de red del host transmisor a la capa de red del host receptor

Tipos de servicios a la capa de red

- ***Servicio sin conexión, sin confirmación de recepción:*** Apropriada cuando la tasa de errores es muy baja. Ejemplo: Ethernet
- ***Servicio sin conexión, con confirmación de recepción:*** Cuando la tasa de errores es importante o en canales inestables Ejemplo: Redes Wi-Fi
- ***Servicio orientado a la conexión con confirmación de recepción:*** Cada trama está numerada, la capa de enlace garantiza el orden y que cada trama será recibida una vez. Ejemplo: Redes WAN

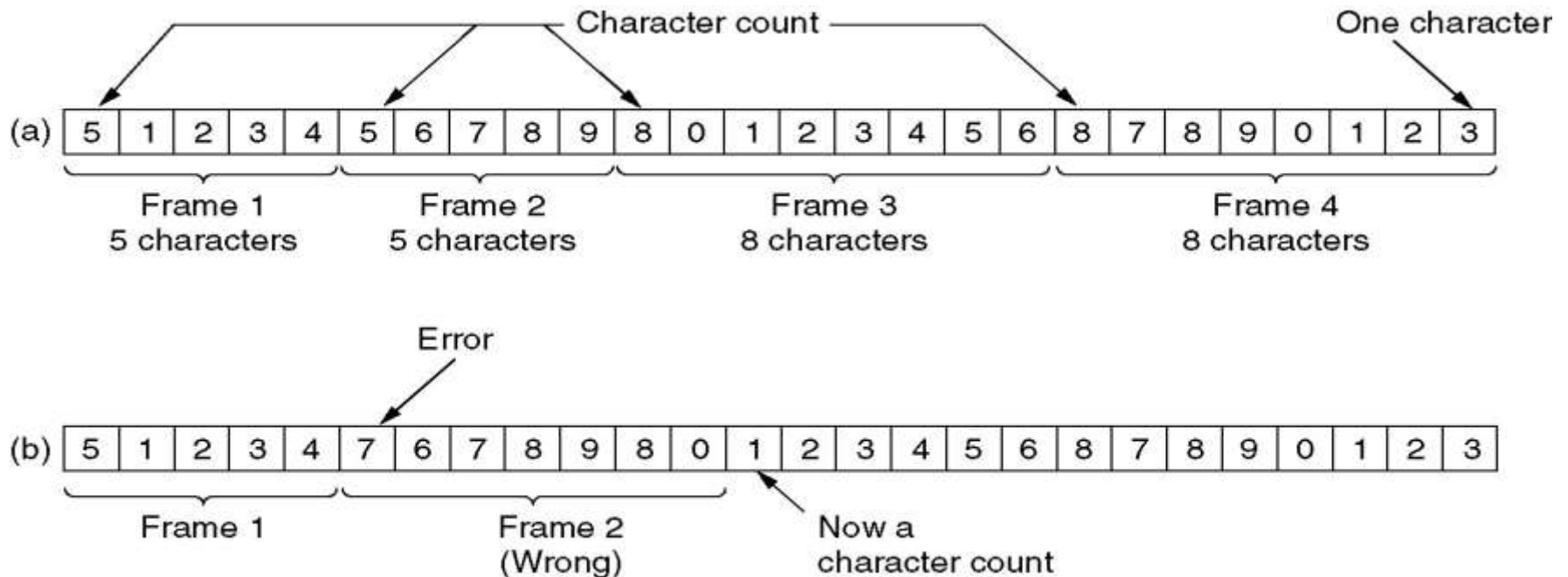
Métodos de identificación de inicio y fin de tramas

- a) Conteo de caracteres o Bytes
- b) Bytes de “Bandera” de inicio y fin, con relleno de bytes
- c) Bits de “Bandera” de inicio y fin, con relleno de bits
- d) Violación de la codificación en la capa física

Usa símbolos para indicar tramas

a) Conteo de caracteres

- Un campo al inicio de la trama indica la longitud de la misma.
- Puede perderse el sincronismo
- Utilizado en combinación con otros métodos



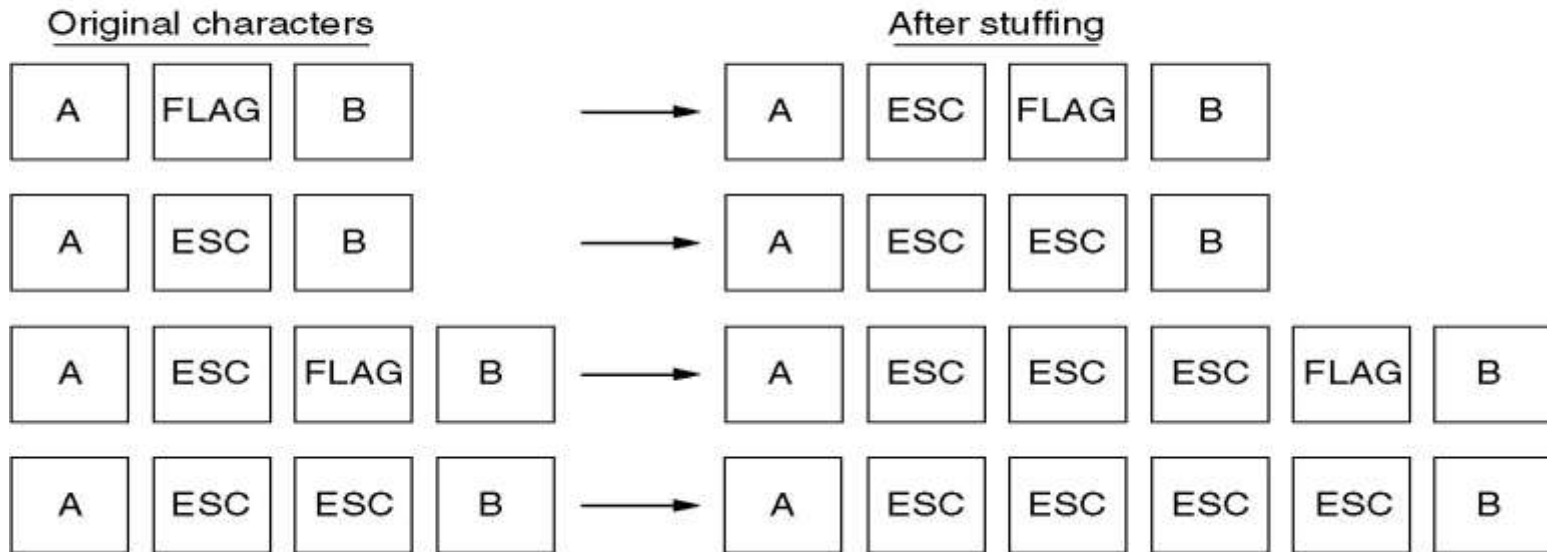
b) Bytes de bandera y relleno de caracteres

- La “bandera” (**01111110**) se adiciona al inicio y al final de una trama. Bytes de “Escape”: indican bytes que no son banderas

Utilizado en ***protocolos orientados a caracteres***



(a)



(b)

c) Bytes de bandera con relleno de bits

(a) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

(b) 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0



Stuffed bits

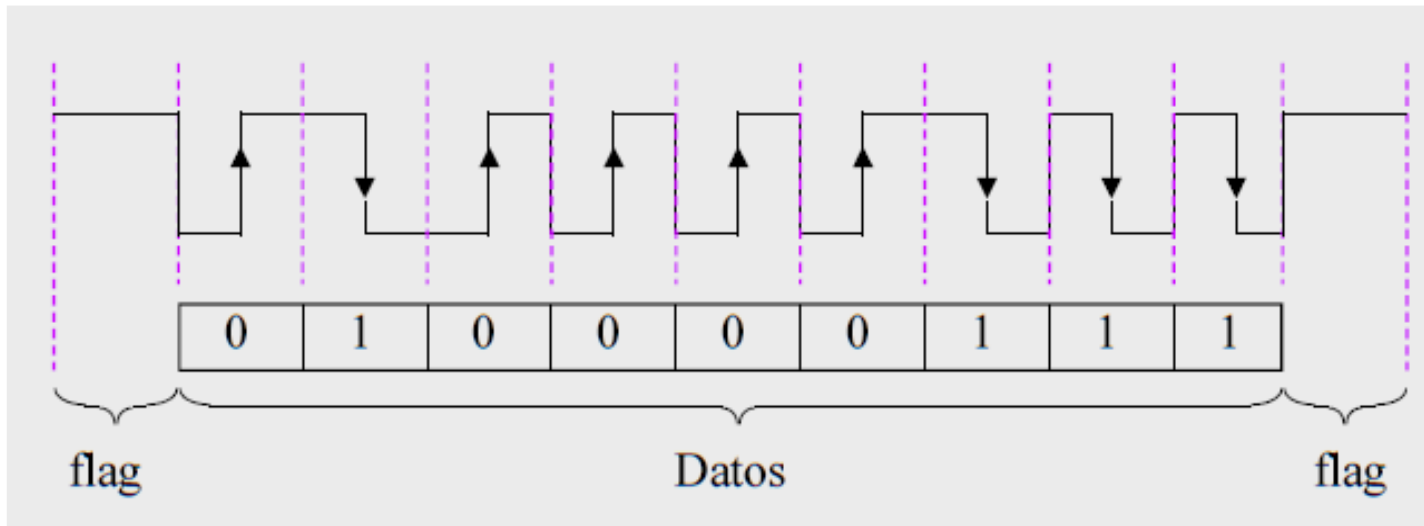
(c) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

Relleno de bits (utilizado en protocolos orientados a bits)

- (a) Datos originales.
- (b) Los datos como aparecen en la línea
- (c) Los datos como son almacenados en la memoria del receptor luego del proceso de des-relleno (destuffing).

d) Violación de códigos de capa física

Ejemplo: En la codificación Manchester, si no existe la inversion bajo-alto o alto-bajo en medio del bit, puede marcar el inicio y final de cada trama



- Asegurar que todas las tramas sean eventualmente recibidas sin errores de transmisión
- Estos errores son debidos a los fenómenos de ruido, distorsión y atenuación en la capa física
- Utilizan códigos (algoritmos) de detección y corrección de errores, basados en la redundancia de bits

Controlar el envío de tramas de transmisión al mayor ritmo en que estas puedan ser aceptadas. Dos métodos:

1. Control de flujo basado en retroalimentación

Ej. Ventanas deslizantes

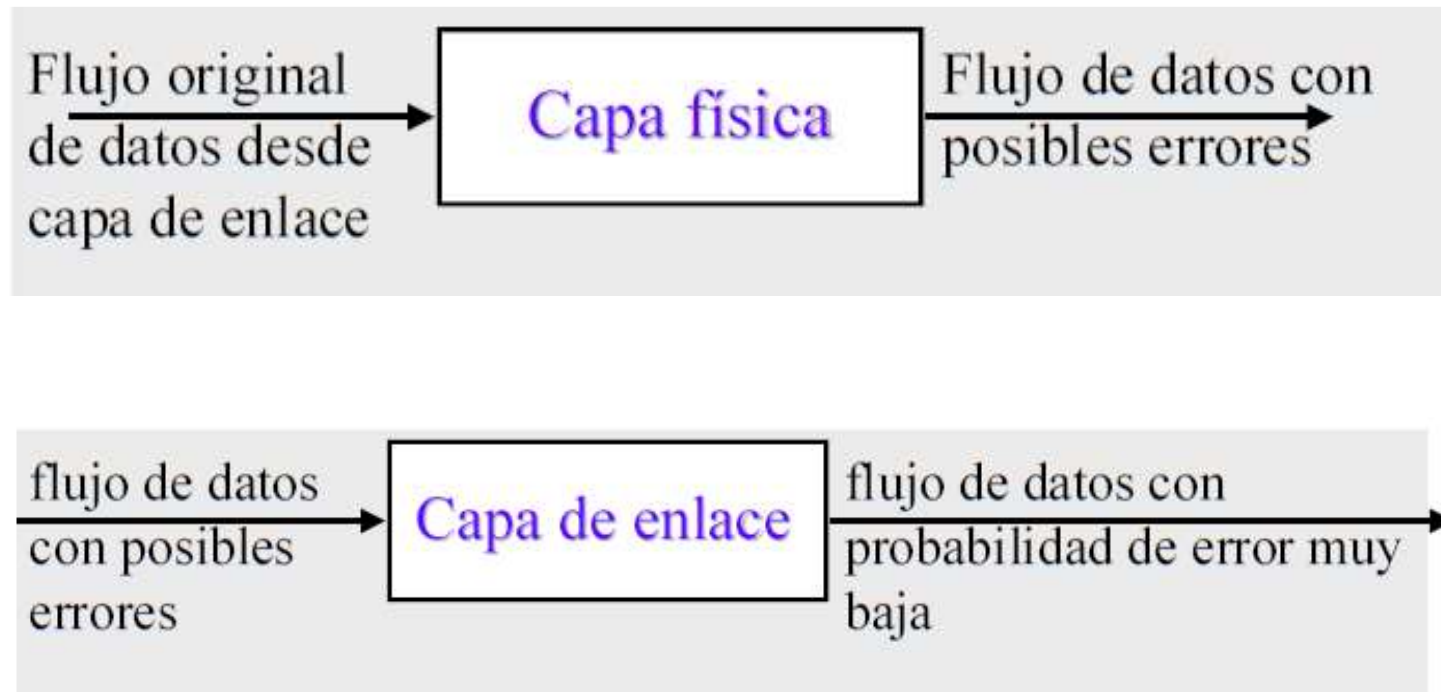
- El receptor envía información al emisor dándole permiso para enviar más datos
- O el receptor le dice al emisor en qué fase está

2. Control de flujo basado en tasas de bits

- El protocolo tiene un mecanismo definido
- Limita la tasa a la cual los emisores pueden transmitir
- No es necesario recibir ningún feedback desde el receptor

3.2 Corrección y Detección de errores

Los **errores de datos** se deben, principalmente, a distorsión, atenuación y ruidos externos en la transmisión de datos.



Códigos de control de errores

Añaden **redundancia estructurada** a los datos (mensaje), para que los errores puedan ser detectados o corregidos.

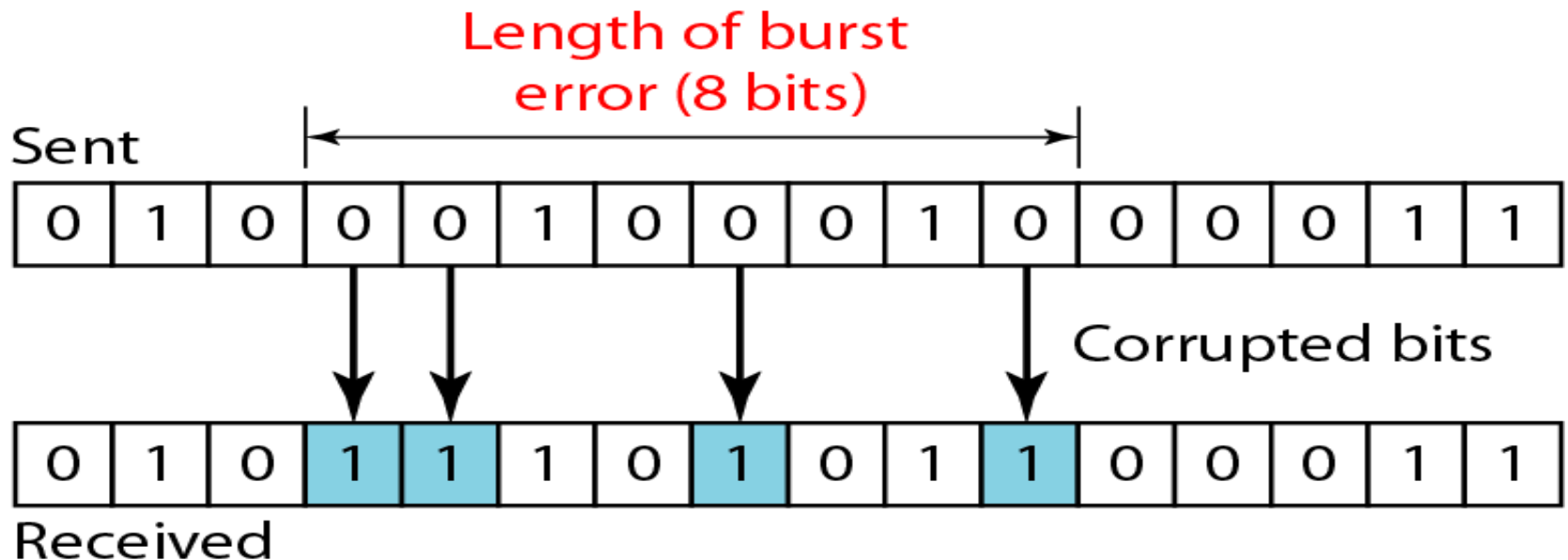
- **Códigos de Corrección de Errores:** Suficiente redundancia para que el receptor pueda deducir cuál era el mensaje correcto
 - FEC (Forward-Error Correction)
 - También aplicados en la capa física
- **Códigos de Detección de Errores:** Menos bits de redundancia solo para deducir que ha ocurrido **un error**
También usualmente aplicados en otras capas



Códigos de control de errores

Una **consideración clave** es el **tipo de error** esperado, ya que ningún método es 100% efectivo

- **Variaciones en el ruido térmico** producen errores uniformemente distribuidos
- **Errores impulsivos** producen errores de **ráfaga**

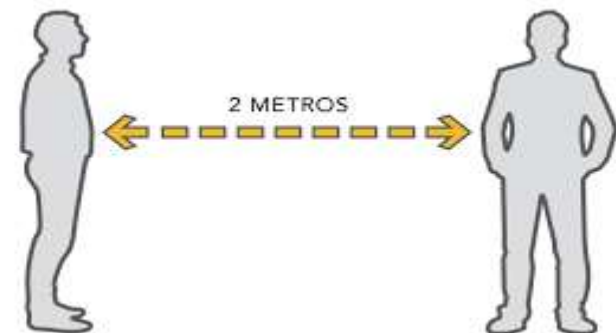


Distancia de Hamming

→ El código de control de errores toma un mensaje de longitud ***m***, le agrega ***r*** bits de redundancia para tener una palabra codificada de longitud igual a ***n*** bits.

$$\mathbf{m + r = n}$$

→ La Distancia de Hamming es la cantidad mínima de bits cambiados para pasar de una **palabra codificada válida** a otra cualquiera.



Ejemplo

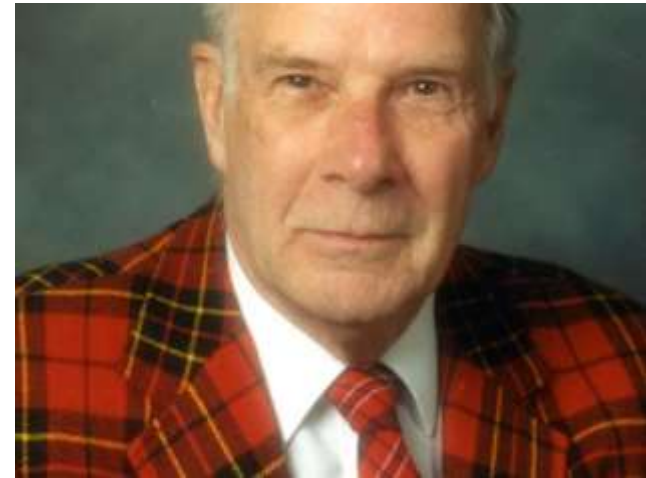
4 palabras codificadas de 10 bits ($m=2$, $r=8$):

0000000000, 0000011111, 1111100000, y 1111111111

- La distancia de Hamming es 5

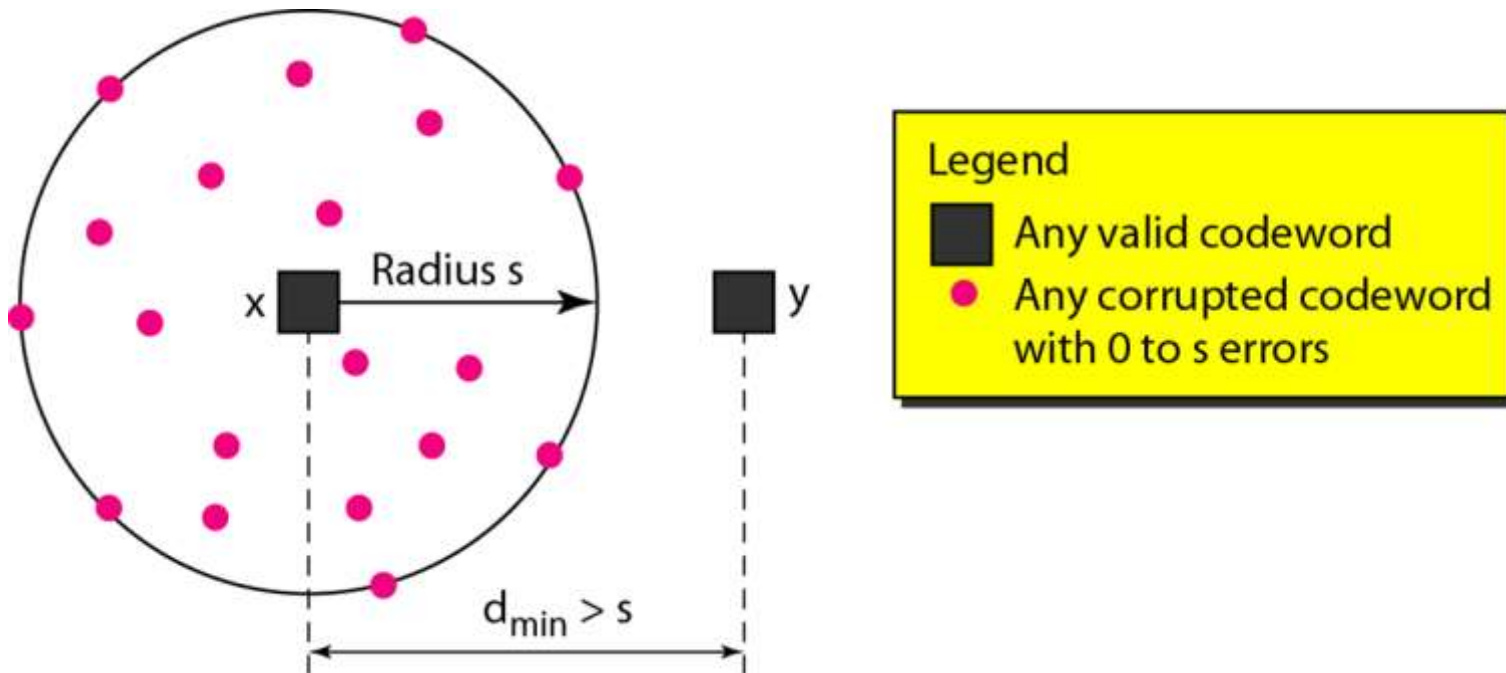
Hamming distance = 3 —

<i>A</i>	1	0	1	1	0	0	1	0	0	1
			↕				↕		↕	
<i>B</i>	1	0	0	1	0	0	0	0	1	1



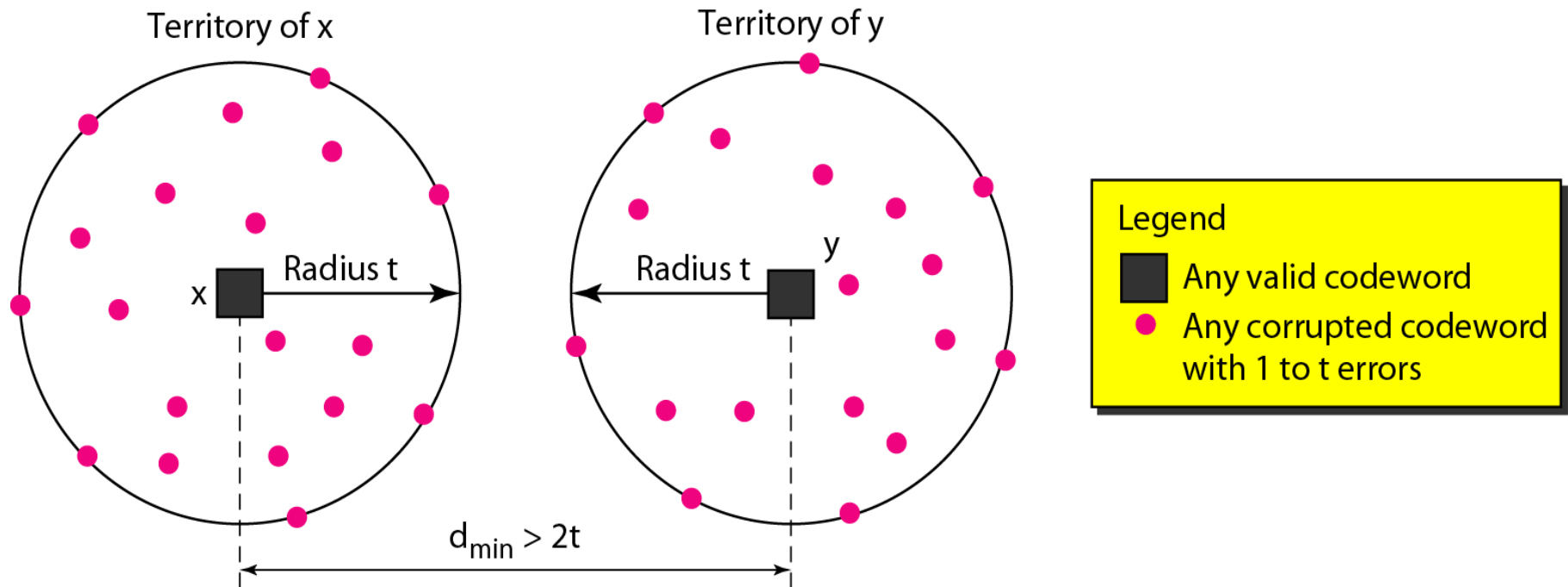
Richard Hamming

Detección de errores y Distancia de Hamming



→ Un código con distancia mínima de Hamming $d_{\min}$ puede **detectar** palabras codificadas (*codewords*) con hasta s bits con error, siendo $d_{\min} > s$

Corrección de errores y Distancia de Hamming



→ Un código con distancia mínima de Hamming $d_{\min}$ puede **corregir** palabras codificadas (*codewords*) con hasta t bits con error, siendo $d_{\min} > 2t$

Corrección de errores de un solo bit

Si tenemos m bits de mensaje y r bits de verificación y queremos corregir todos los errores de un bit;

- Se tienen 2^m mensajes posibles.
- Cada mensaje necesita **($m+r+1$)** palabras codificadas, para indicar: la ausencia de error (una palabra codificada válida) o para indicar la ubicación del error ($m+r$ posibilidades)
- Se pueden tener $2^n = 2^{m+r}$ palabras codificadas

$$2^m \cdot (m+r+1) \leq 2^{m+r} = 2^m \cdot 2^r$$

$$(m + r + 1) \leq 2^r$$



Código de Hamming

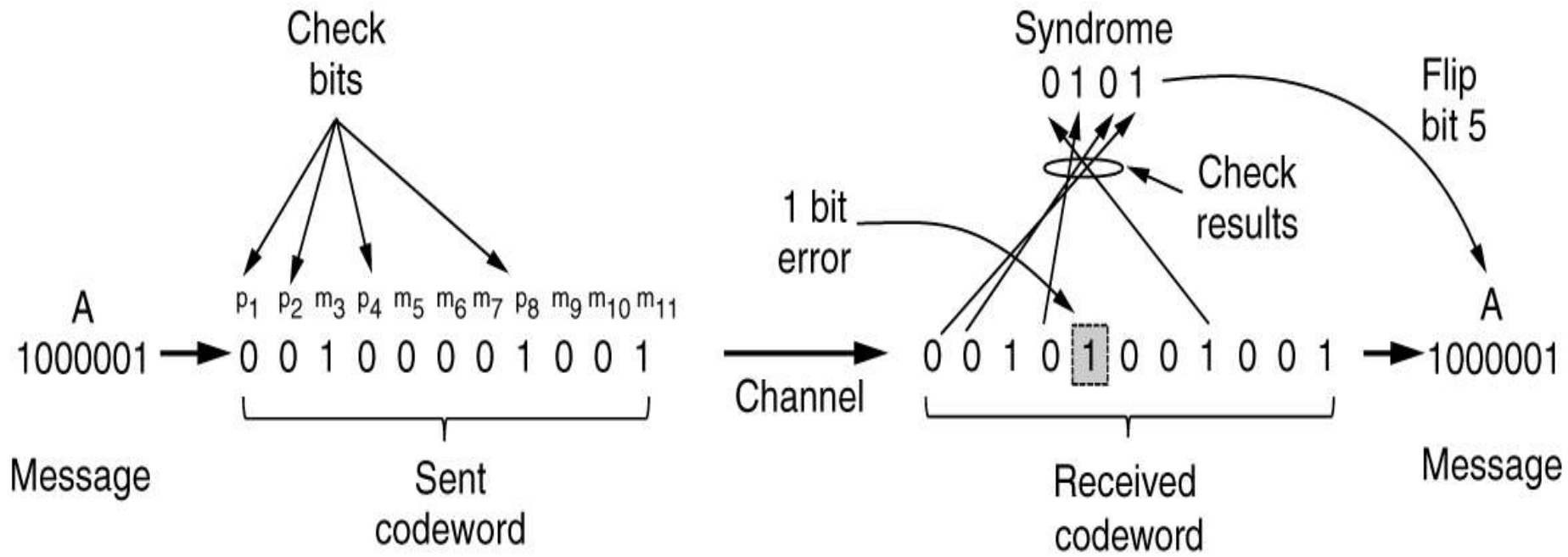
Método simple para añadir bits de verificación y corregir
ERRORES SIMPLES de hasta un bit:

- A través de bits de paridad sobre sub-conjuntos de la palabra codificada
- El recómputo de las sumas de paridad (syndrome) proporciona la posición del bit con error, o “0” si no existe error.
- Los sub-conjuntos se obtienen considerando sus posiciones (potencias de 2)



Ejemplo

Ejemplo del Código de Hamming para un mensaje de 7 bits



Mensaje $m = 00010011001$

	P1	P2	3	P4	5	6	7	P8	9	10	11	12	13	14	15
Bits			0		0	0	1		0	0	1	1	0	0	1
P1	1		0		0		1		0		1		0		1
P2		1	0			0	1			0	1			0	1
P4				1	0	0	1					1	0	0	1
P8								1	0	0	1	1	0	0	1
Bits (con paridad)	1	1	0	1	0	0	1	1	0	0	1	1	0	0	1

	P1	P2	3	P4	5	6	7	P8	9	10	11	12	13	14	15
Bits			0		0	0	1		0	0	1	1	0	0	1
P1	1		0		0		1		0		1		0		1
P2		1	0			1	1			0	1			0	1
P4				1	0	1	1					1	0	0	1
P8								1	0	0	1	1	0	0	1
Bits (con paridad)	1	1	0	1	0	1	1	1	0	0	1	1	0	0	1

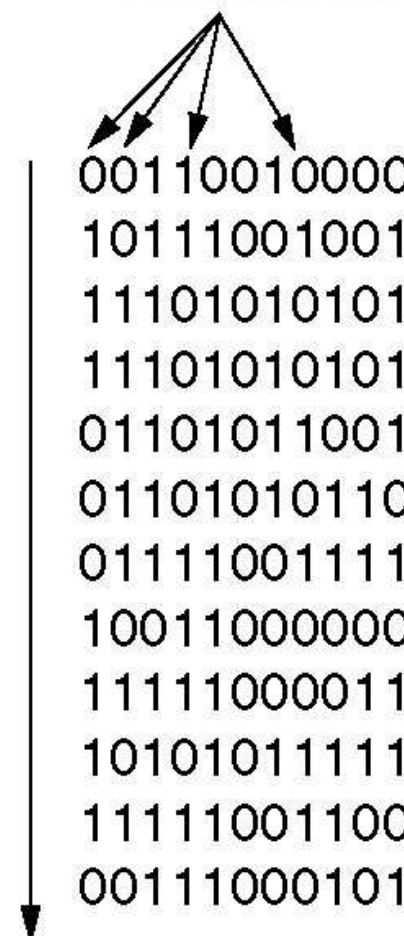
✓

✗

✗

✓

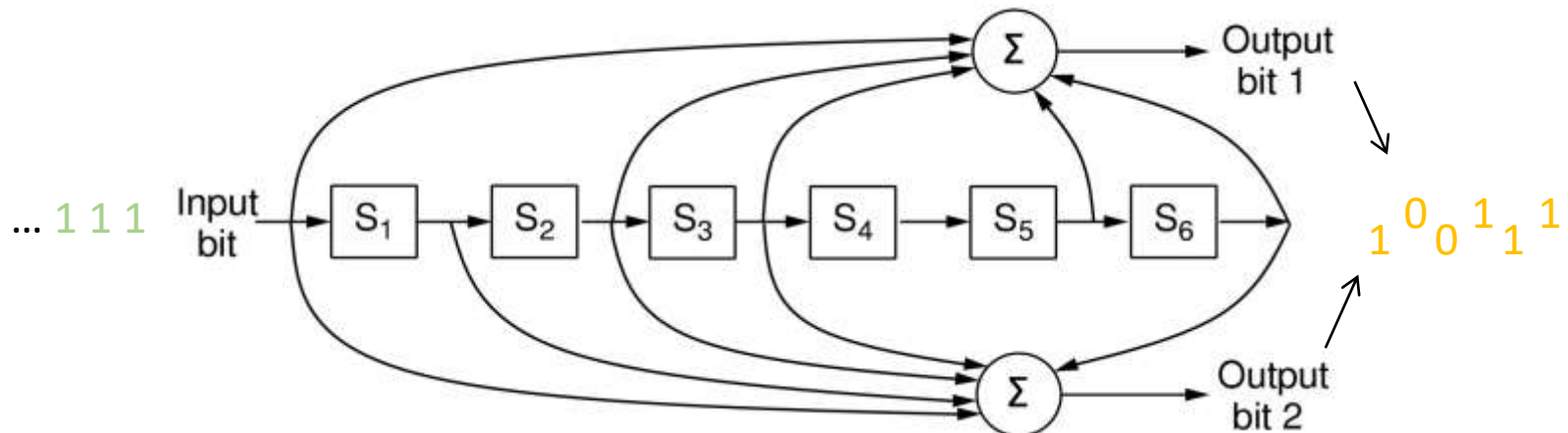
Código de Hamming en errores de ráfaga

Char.	ASCII	Check bits
		
H	1001000	00110010000
a	1100001	10111001001
m	1101101	11101010101
m	1101101	11101010101
i	1101001	01101011001
n	1101110	01101010110
g	1100111	01111001111
	0100000	10011000000
c	1100011	11111000011
o	1101111	10101011111
d	1100100	11111001100
e	1100101	00111000101
		Order of bit transmission

Códigos convolucionales

Operan en una cadena de bits, manteniendo el estado interno

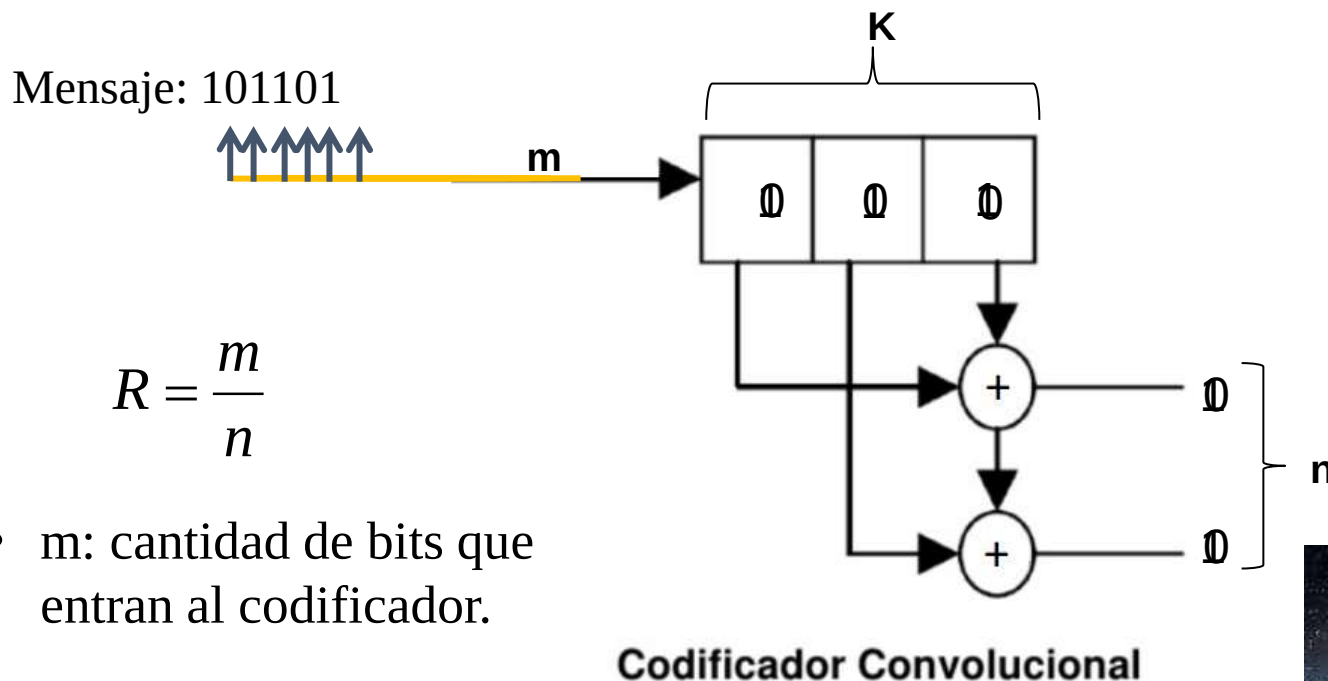
- La salida es una función de todos los bits de entrada anteriores
- Los Bits son decodificados con el “Algoritmo de Viterbi”



Código binario convolucional de la NASA (tasa = $\frac{1}{2}$)
usado en IEEE 802.11

Códigos Convolucionales

- Codificación



Mensaje Codificado:

110100101000

$$R = \frac{m}{n}$$

- m : cantidad de bits que entran al codificador.
- n : cantidad de bits que salen del codificador.

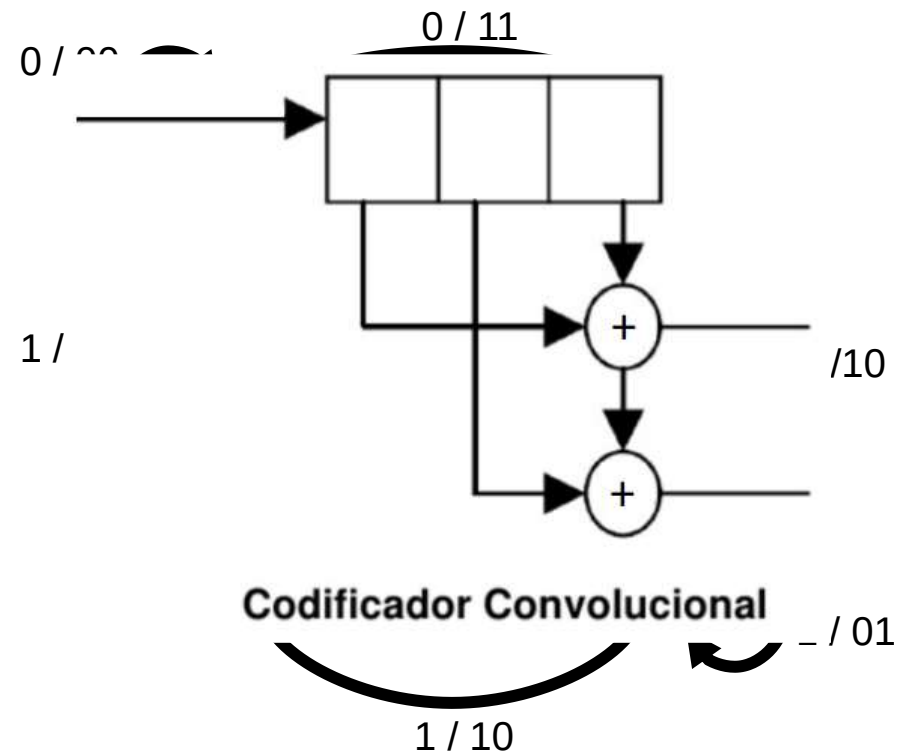


Códigos Convolucionales

- Tabla de Transiciones

Entrada	S_o	S_f	Salida
0	00	00	00
1	00	10	11
0	01	00	11
1	01	10	00
0	10	01	01
1	10	11	10
0	11	01	10
1	11	11	01

- Diagrama de Estados

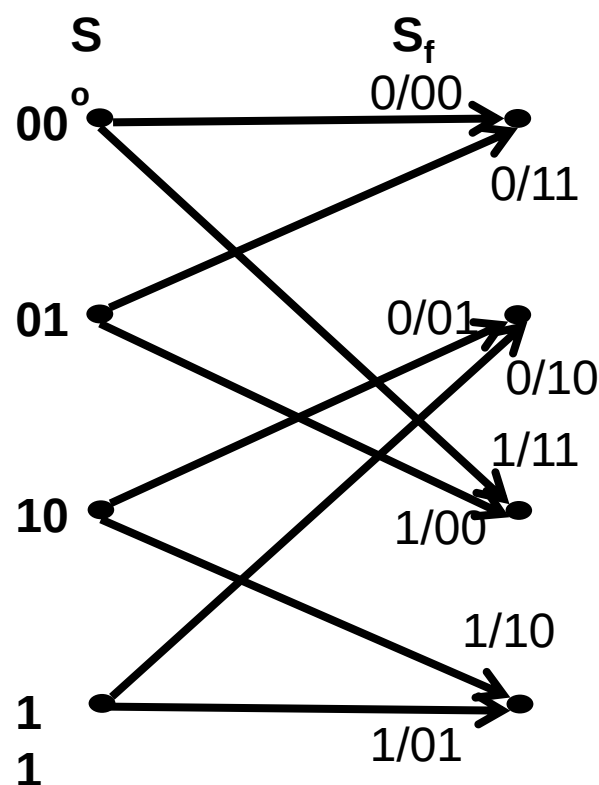


Códigos Convolucionales

- Tabla de Transiciones

Entrada	S_o	S_f	Salida
0	00	00	00
1	00	10	11
0	01	00	11
1	01	10	00
0	10	01	01
1	10	11	10
0	11	01	10
1	11	11	01

- Diagrama de Trellis



Algoritmo de Viterbi

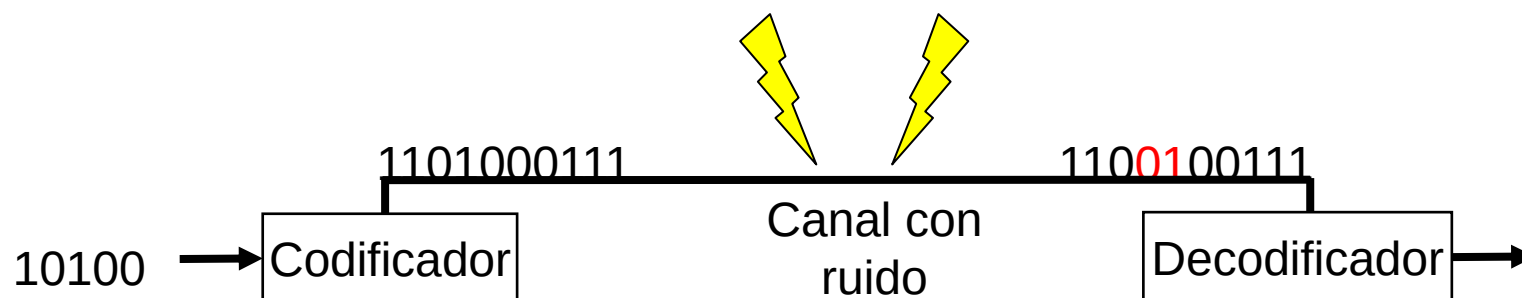
Algoritmo utilizado en la decodificación de Códigos Convolucionales en canales con ruido.

Entrada:

Mensaje codificado + error.

Salida:

Secuencia de estados con mayor probabilidad de generar el mensaje recibido.



Algoritmo de Viterbi-Visualización del Algoritmo

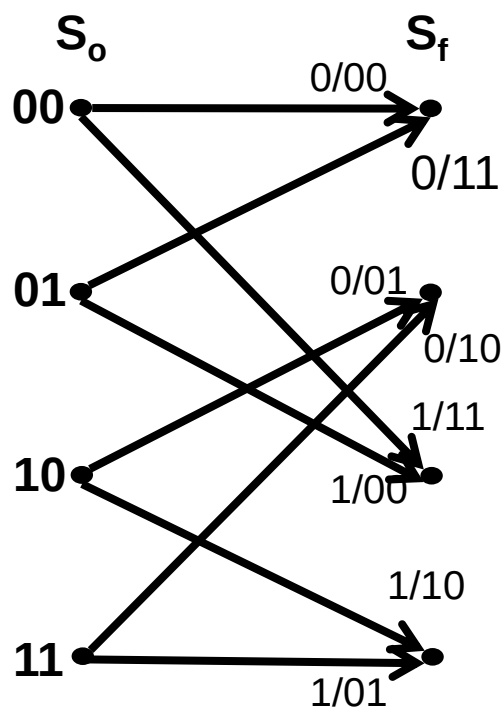
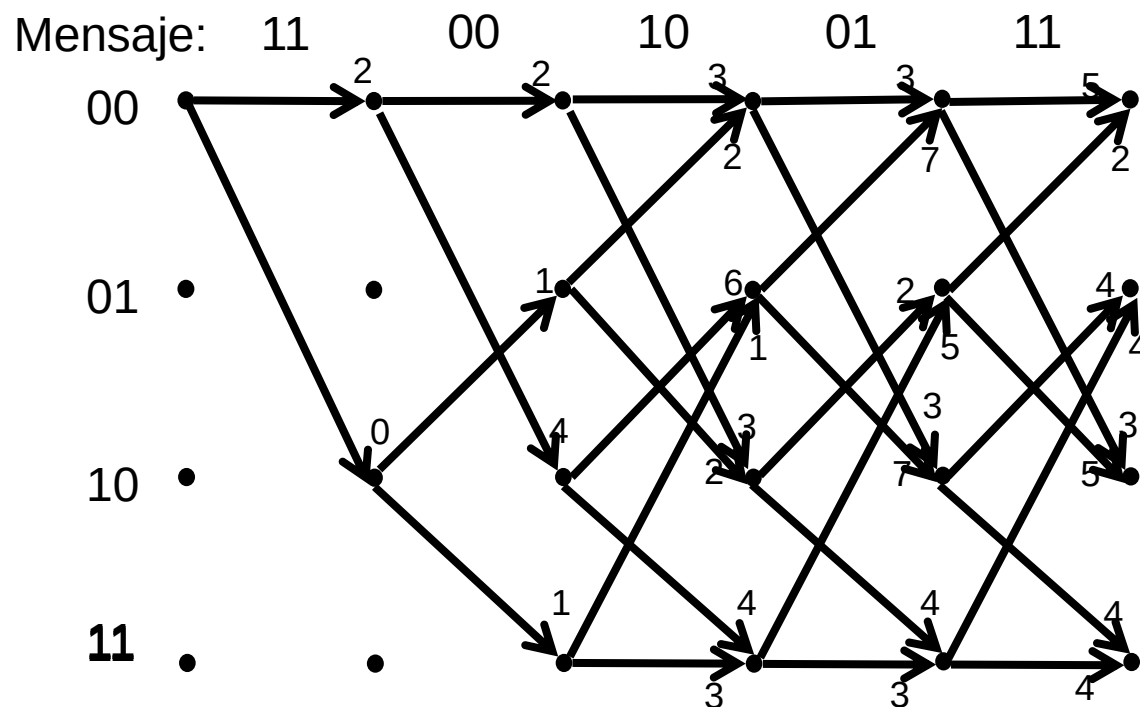


Diagrama de Trellis del codificador



Mensaje Decodificado: 10100

Algoritmo de Viterbi

- La elección del número de generadores en el Codificador Convolutacional es importante.
- El Algoritmo de Viterbi no es usado exclusivamente en la decodificación de los Códigos Convolucionales. Tiene varias aplicaciones en:
 - Reconocimiento de Voz
 - Sintetización de Voz
 - Bioinformática



►→ Reed-Solomon codes

Códigos de bloques lineales

A menudo sistemáticos

Basados en el hecho de que todos los polinomios de grado n son determinados completamente por $n+1$ puntos

►→ LDPC (Low-Density Parity Check) codes

Códigos de bloques lineales

Cada bit de salida está formado por solo una fracción de los bits de entrada

Práctico para tamaños de bloque grande

Tiene habilidades de corrección de errores que supera a muchos códigos de error

Detección de errores: Bits de paridad

Consiste en agregar **un solo bit de redundancia** al mensaje

Para que la suma de bits 1 sea par (paridad par) o impar (paridad impar)

- Equivalente a XOR; (paridad par)
- Si $m = 1110000 \rightarrow n = 1110000 \mathbf{1}$
- **Distancia de Hamming de la codificación = 2**

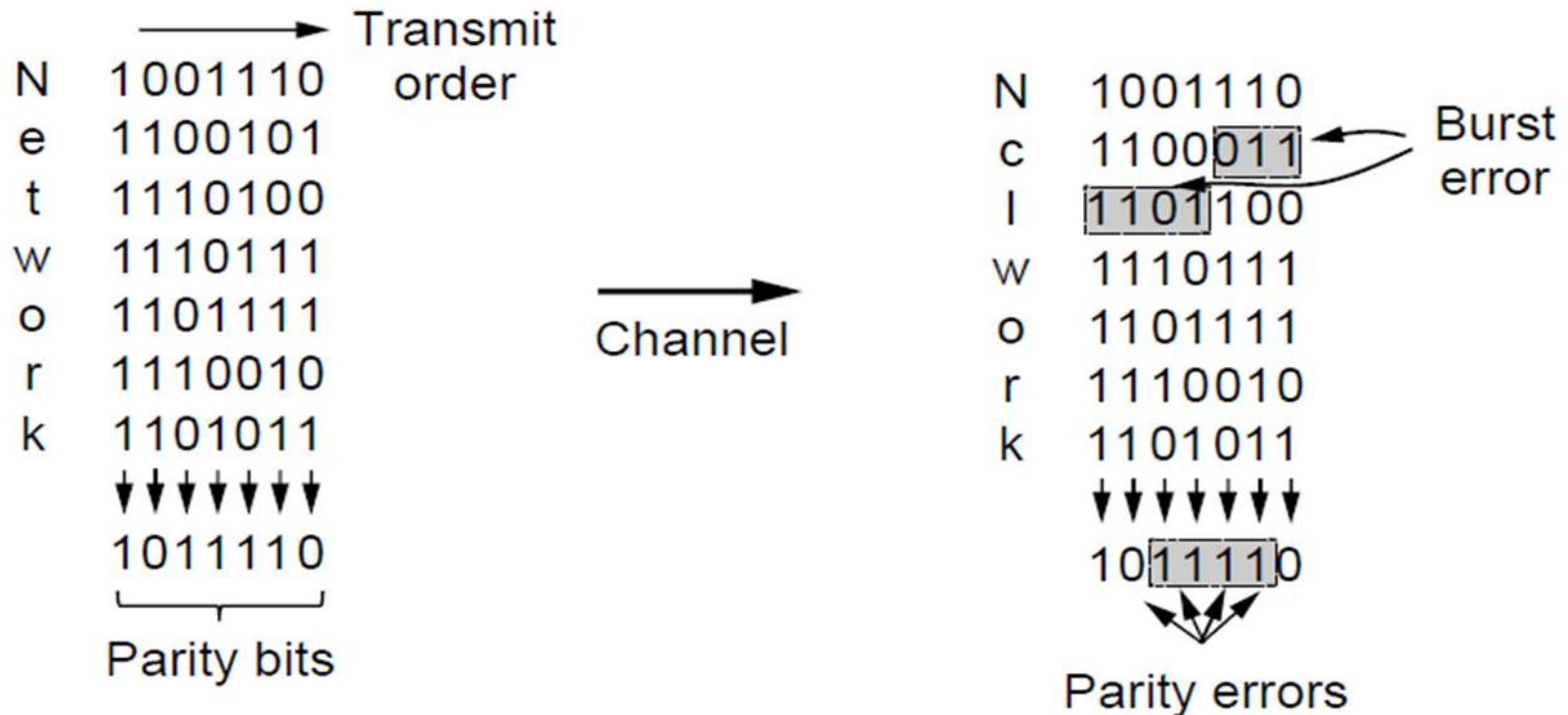
Se detecta **errores con un número impar** de bits

- Ej: 1 bit, 1110010 $\mathbf{1}$; detected
- Ex: 3 bits, 1101100 $\mathbf{1}$; detected
- Ex: 2 bits, 1110110 ; *not detected*
- El error puede estar en el bit de paridad mismo

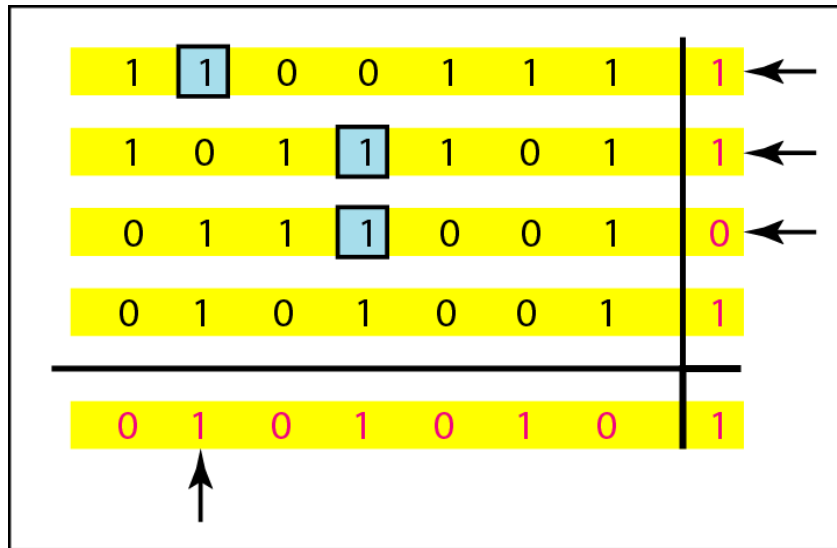
Ráfagas de error

Interponiendo N bits de paridad detecta errores de ráfaga de longitud hasta de N

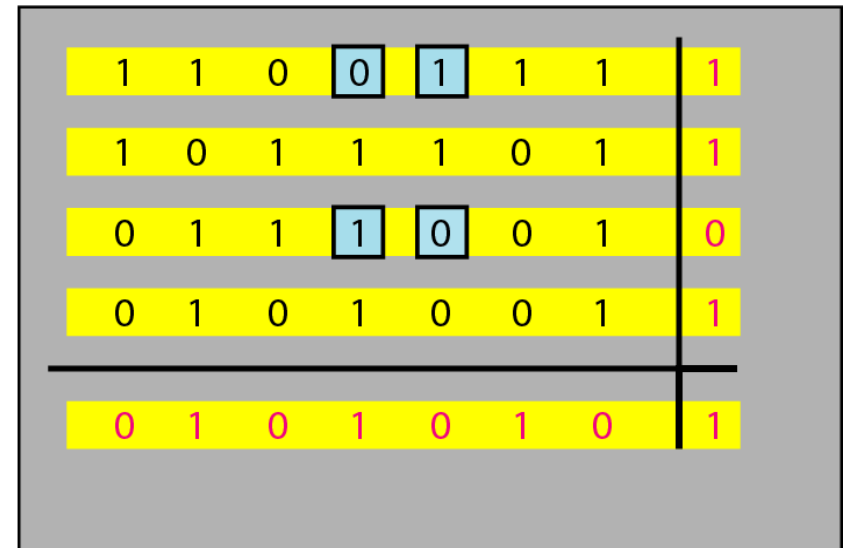
- Cada suma de paridad se realiza sobre bits no adyacentes



Bits de paridad en dos dimensiones



d. Three errors affect four parities



e. Four errors cannot be detected

Checksums o sumas de comprobación

El Checksum trata datos como palabras de longitud N y añade N bits de redundancia igual a la **suma modulo 2^N** de las palabras

Propiedades:

- Mejor detección de errores que los bits de paridad
- Detecta ráfagas de hasta N errores
- Vulnerable a errores sistemáticos, ej. adición de ceros



Ejemplo con números binarios

Calcular el checksum de 8 bits del mensaje m= 0xA037B6

	1	101	11	
0x A0	1010	0000		
0x 37	0011	0111		
0x B6	1011	0110		
	=====			
	1000	1101		
			1	
	=====			
	1000	1110		
	0111	0001	checksum	0x71

	1	101	111	
0xA0	1010	0000		
0x37	0011	0111		
0xB6	1011	0110		
0x71	0111	0001		
	=====			
		1111	1110	
				1
	=====			
		1111	1111	
		0000	0000	

Ejemplo con números hexadecimales

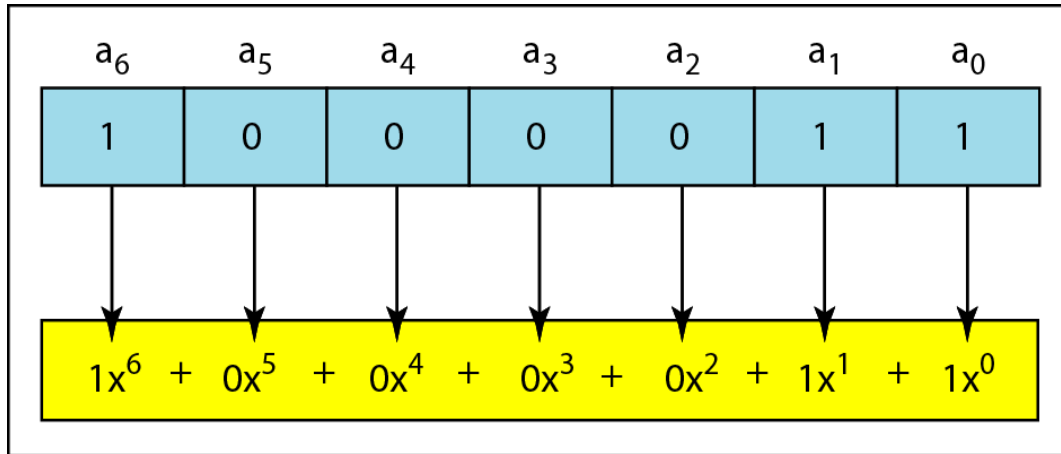
1	0	1	3	Carries	
4	6	6	F	(Fo)	
7	2	6	F	(ro)	
7	5	7	A	(uz)	
6	1	6	E	(an)	
0	0	0	0	Checksum (initial)	
8	F	C	6	Sum (partial)	
8	F	C	7	Sum	
7	0	3	8	Checksum (to send)	

a. Checksum at the sender site

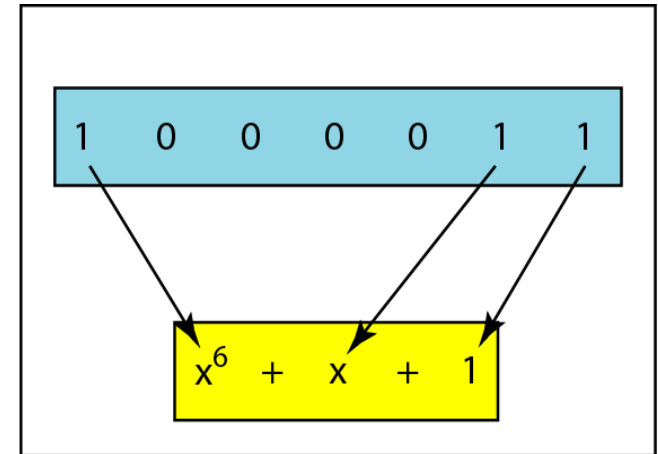
1	0	1	3	Carries	
4	6	6	F	(Fo)	
7	2	6	F	(ro)	
7	5	7	A	(uz)	
6	1	6	E	(an)	
7	0	3	8	Checksum (received)	
F	F	F	E	Sum (partial)	
F	F	F	F	Sum	
0	0	0	0	Checksum (new)	

a. Checksum at the receiver site

Redundancia cíclica (CRC)



a. Binary pattern and polynomial



b. Short form

- Los códigos CRC (*Cyclic Redundance Check*) relacionan los mensajes y las palabras codificadas con polinomios de coeficiente “0” o “1”.
- El algoritmo se basa en verificar el resto de una división con un “**polinomio generador**” $G(x)$, conocido por el emisor y el receptor

Cálculo del CRC

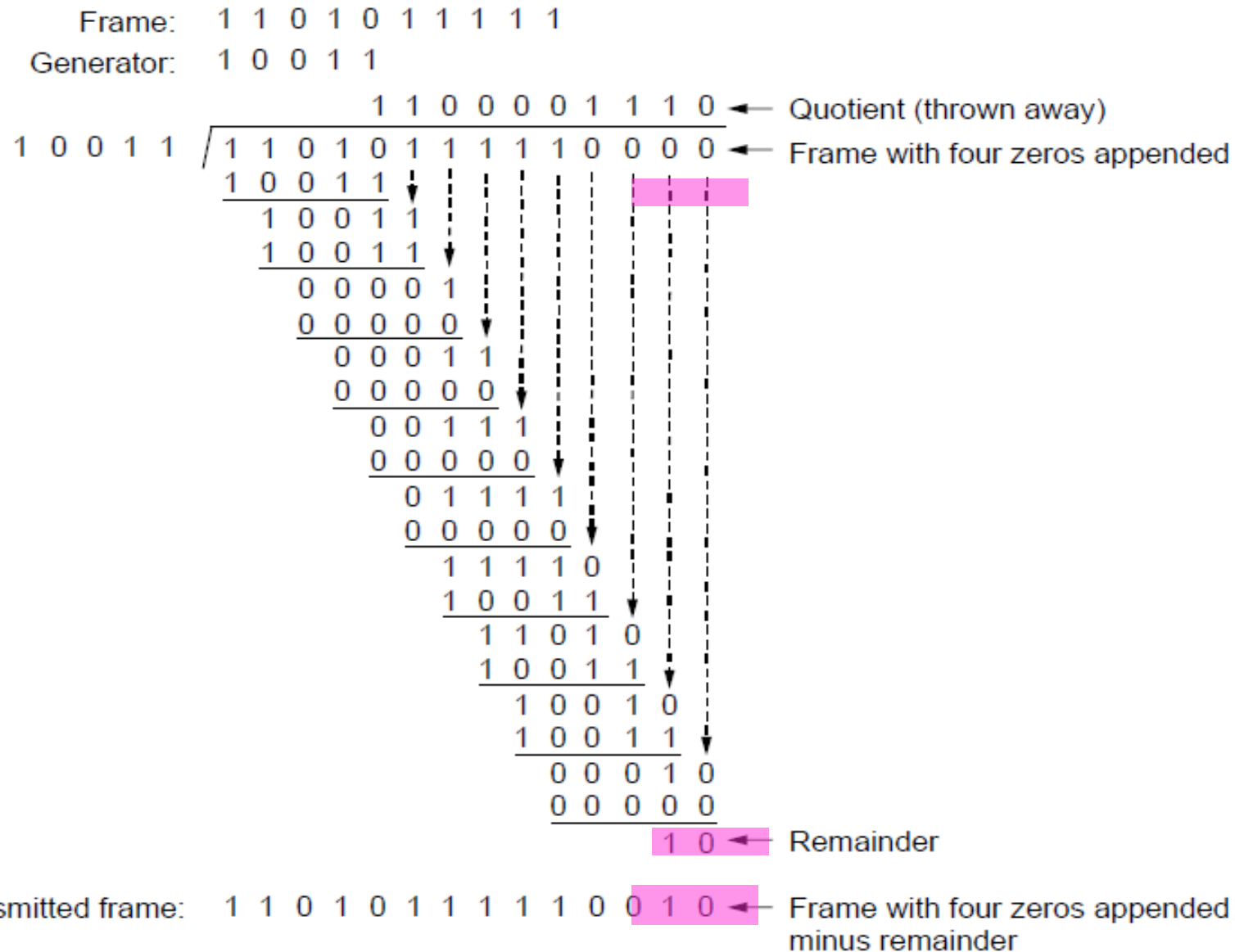
En el EMISOR

- 1.- Si r es el grado de $G(x)$, anexe r bits “0” a $m(x)$, esto equivale a obtener $m(x) \cdot x^r$
- 2.- Divida $m(x) \cdot x^r$ entre $G(x)$ modulo 2
- 3.- Reste el residuo res (de r o menos bits) del dividendo, obteniendo $T(x)$, la palabra codificada divisible entre $G(x)$. Esto equivale a reemplazar algunos, o todos los bits “0” anexados en el paso 1, por res .

En el RECEPTOR

- 1.- Divida $T(x)$ entre $G(x)$.
- 2.- Si el residuo es igual a 0, se considera que no hubo errores. De lo contrario se rechaza la trama.

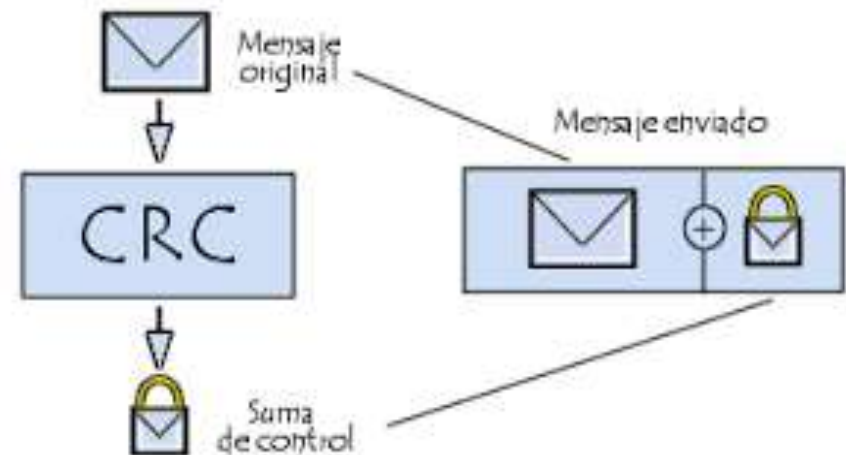
Ejemplo



Errores de transmisión

- Si ocurre un error de transmisión, se recibe: $T(x) + E(x)$
Cada bit “1” de $E(x)$ corresponde a un error.
- En el receptor se calcula realmente: $(T(x) + E(x)) / G(x)$
Como $T(x) / G(x)$ es exacto
- El resto de esta división será igual al de: $E(x) / G(x)$

→ Todos los errores $E(x)$ que sean divisibles entre $G(x)$ no serán detectados



Análisis de $G(x)$

→ $E(x) = x^i$ siendo **i** la ubicación del bit con error

Si la palabra codificada es: 100 0100 0101 0001 0011

Y el mensaje recibido es: 100 0100 0101 00**1**1 0011

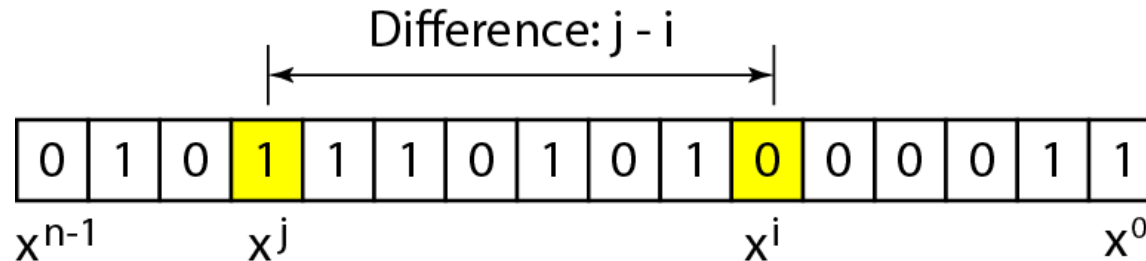
$E(x)$ es: 000 0000 0000 0010 0000

$$E(x) = x^5$$

Si $G(x)$ tiene más de un término, $E(x)/G(x)$ nunca será exacto.

Si $G(x)$ tiene más de un término distinto a cero, detectará todos los errores de un bit.

Errores de dos bits



$$E(x) = x^i + x^j = (x^{j-i} + 1) x^i = (x^t + 1) x^i$$

Si $G(x)$ no divide a $x^t + 1$; $r < t < n - 1$, todos los errores dobles aislados serán detectados.

Se deben realizar varias divisiones de x^t+1 entre $G(x)$ para todos los valores posibles de t . Si la division es inexacta, significa que se detectarán todos los errores dobles a esa distancia t entre los bits con error.

Errores con número impar de bits

→ Un Generador $G(x)$ que contiene un factor igual a $(x + 1)$ puede detectar todos los errores impares:

→ $(x+1)$ no divide a ningún polinomio con número impar de términos, ej. $x^7+x^6+x^2$

Si al realizar la division de $G(x)$ entre $(x+1)$ el resto es cero, todos los errores con número impar de bits serán detectados



Errores de ráfaga

- Todas las ráfagas de error con $L \leq r$ serán detectadas si $G(x)$ contiene un término x^0

$$E(x) = x^i (x^k + x^{k-1} + \dots + 1)$$

El término independiente de $G(x)$ debe tener el valor “1” a fin de mejorar la detección de ráfagas de error.



Análisis del polinomio generador

1. Debe tener al menos dos términos (para detectar todos los errores de un bit).
2. El coeficiente del término x^0 debe ser 1 (para mejorar la detección de errores de ráfaga).
3. No debe dividir $x^t + 1$, para t entre 2 y $n - 1$
4. Debe tener el factor $(x + 1)$ para detectar todos los errores con número impar de bits.

<i>Name</i>	<i>Polynomial</i>	<i>Application</i>
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs